



INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo



Práctica 9

CONTROL DE UN SERVOMOTOR

Sistemas en Chips

- ▶ **Prof. José Juan Pérez Pérez**
- ▶ **Grupo: 6CVI**
- ▶ **Equipo I:**
 - ▶ Ayala Fuentes Sunem Gizeht
 - ▶ Guevara Badillo Areli Alejandra
 - ▶ León Pérez May Carolina

▶ viernes, 7 de enero de 2026

Introducción

Funcionamiento de un servomotor

El servomotor se controla mediante una señal digital llamada **PWM**. A diferencia de un motor de corriente continua (DC) que gira continuamente, el servo interpreta la **duración** de un pulso eléctrico como una instrucción de posición específica.

Retrasos por software

Un Timer es un periférico interno que cuenta ciclos de reloj de forma independiente a la Unidad Central de Procesamiento (CPU). Esto permite medir el tiempo o generar eventos periódicos sin que la CPU tenga que estar "ocupada" ejecutando bucles de retardo (*busy-waiting*).

Para poder controlar adecuadamente ciertos elementos de un circuito o realizar operaciones de manera lenta, se implementa una rutina de retraso (en este caso medms) que al ser llamada se genera un retraso de 500ms.

Esta nos será útil para poder controlar el servomotor variando la duración del pulso eléctrico.

(Overflow):

- El contador inicia en 0 y cuenta hacia arriba hasta su valor máximo (255 para 8 bits, 65535 para 16 bits).
- Al pasar del máximo a 0 nuevamente, se genera una bandera de "Desbordamiento" (Overflow Flag).
- Si la interrupción está habilitada, el código salta a la rutina correspondiente.

Planteamiento del ejercicio

El objetivo de la práctica consiste en diseñar e implementar un sistema de control para un servomotor controlada por el microcontrolador **ATmega8535**.

El sistema debe posicionar el servomotor en 3 diferentes ángulos usando botones

Las especificaciones funcionales son:

1. **Control:** Usar botones para posicionar el servomotor.
2. **Movimiento del servomotor:** Rotar el servomotor a 0, 90 y 180 grados.

Desarrollo

Material

Para la realización de esta práctica se requieren los siguientes componentes y herramientas:

- Microcontrolador ATmega8535.
- 3 Pulsadores (*Push-buttons*).
- 1 servomotor
- Fuente de alimentación de 5V DC.
- 1 placas de pruebas.
- Cables de conexión.

Código

```
.include "m8535def.inc"
.def aux = r16

; Definición de la macro para generar PWM
.macro pulso
    sbi porta,0      ; Pone en alto el pin 0 del puerto A
    ldi aux,@0       ; Carga el primer valor (tiempo en alto)
uno:
    rcall medms      ; Llama a la rutina de retardo
    dec aux          ; Decrementa el contador
    brne uno         ; Si no es cero, repite
    cbi porta,0      ; Pone en bajo el pin 0 del puerto A
    ldi aux,@1       ; Carga el segundo valor (tiempo en bajo)
cta:
    rcall medms      ; Llama a la rutina de retardo
    dec aux          ; Decrementa el contador
    brne cta         ; Si no es cero, repite
.endm

    ldi aux,low(ramend)
    out spl,aux
    ldi aux,high(ramend)
    out sph,aux

    ser aux          ; Pone aux en 0xFF (todos unos)
    out ddra,aux     ; Configura Puerto A como Salida
    out portd,aux    ; Activa resistencias Pull-up en Puerto D
```

```

; Bucle principal de chequeo de botones
checa:
    sbis pind,5    ; Salta si el bit 5 del Puerto D está en 1 (botón no presionado)
    rcall cero     ; Si es 0 (presionado), llama a rutina 'cero'
    sbis pind,6    ; Checa bit 6
    rcall noventa  ; Si presionado, llama a 'noventa'
    sbis pind,7    ; Checa bit 7
    rcall cien80   ; Si presionado, llama a 'cien80'
    rjmp checa     ; Repite el ciclo

; Rutinas de posición del servo
cero:
    pulso 2,38     ; Macro con parámetros para 0 grados
    ret

noventa:
    pulso 3,37     ; Macro con parámetros para 90 grados
    ret

cien80:
    pulso 4,36     ; Macro con parámetros para 180 grados
    ret

; Rutina de retardo (Delay)
medms:
    ldi r18, 164
L1: dec r18
    brne L1
    nop
    ret

```

Ilustración 1. Código implementado.

Solución

La solución implementada utiliza los siguientes procedimientos:

1. **Variación de pulso (PWM):** Se definen los valores que dura el pulso, a más duración más el ángulo. Se utiliza la rutina de medio milisegundo de retardo (medms) para poder controlar la longitud del tiempo en high y en low. Cada periodo debe durar lo mismo.
2. **Control mediante botones:** Se usan los botones para seleccionar cada uno de los ángulos (0, 90 | 180).

Circuito

La interfaz de hardware se establece de la siguiente manera:

- **Puerto A pin 0 (Pin de control):** Se usa para controlar el servomotor.
- **Puerto D pin 5-7(Entradas de control):** Conectado a los botones que controlan el ángulo del servomotor

Diagrama del circuito

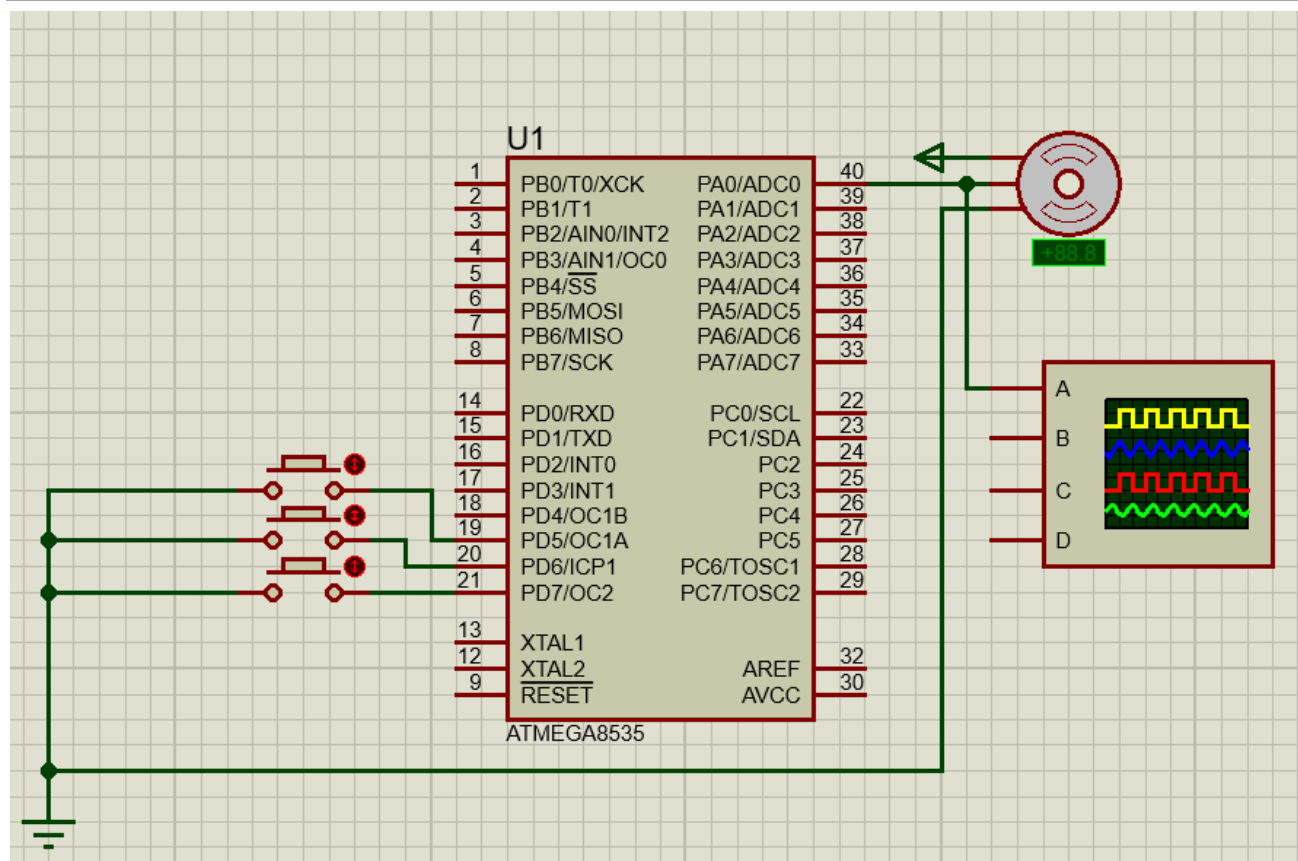
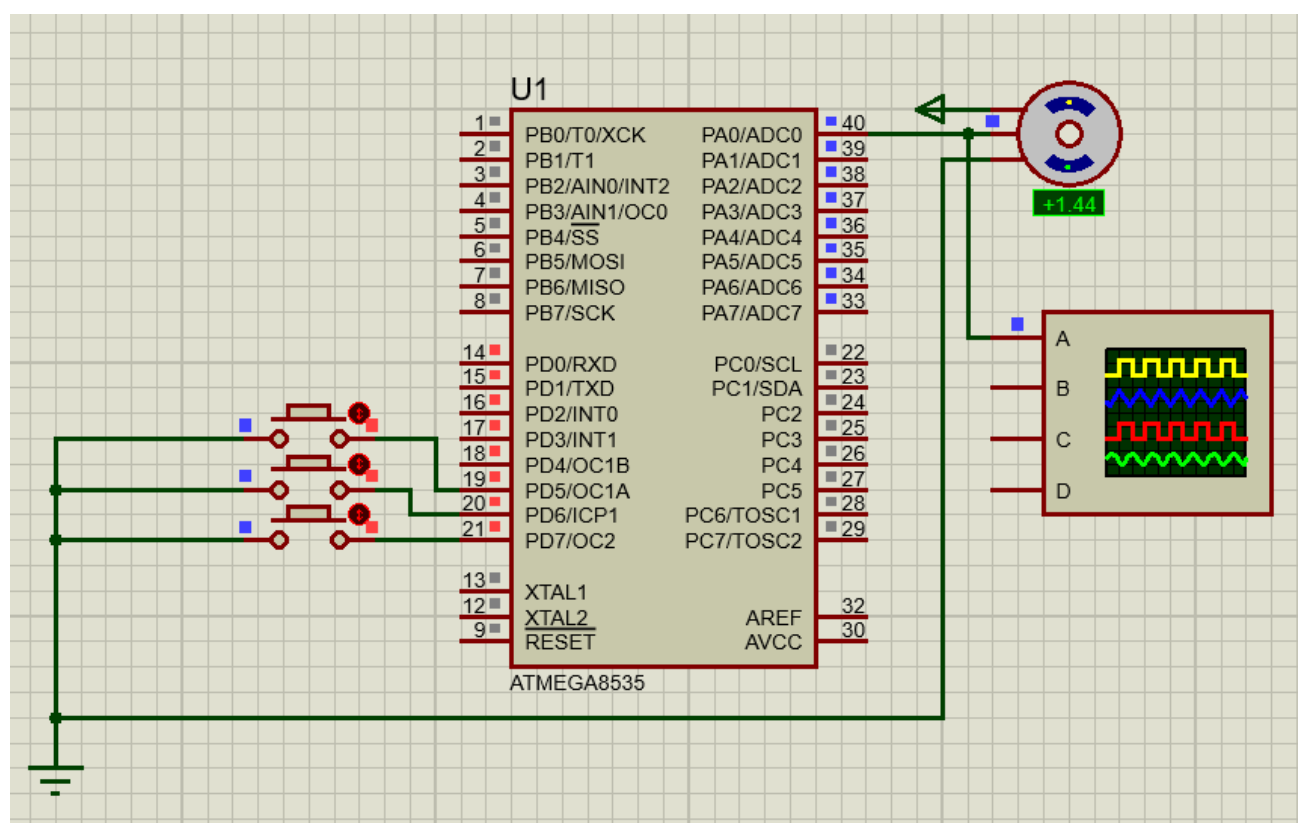


Ilustración 2. Esquema del circuito en proteus.

Simulación



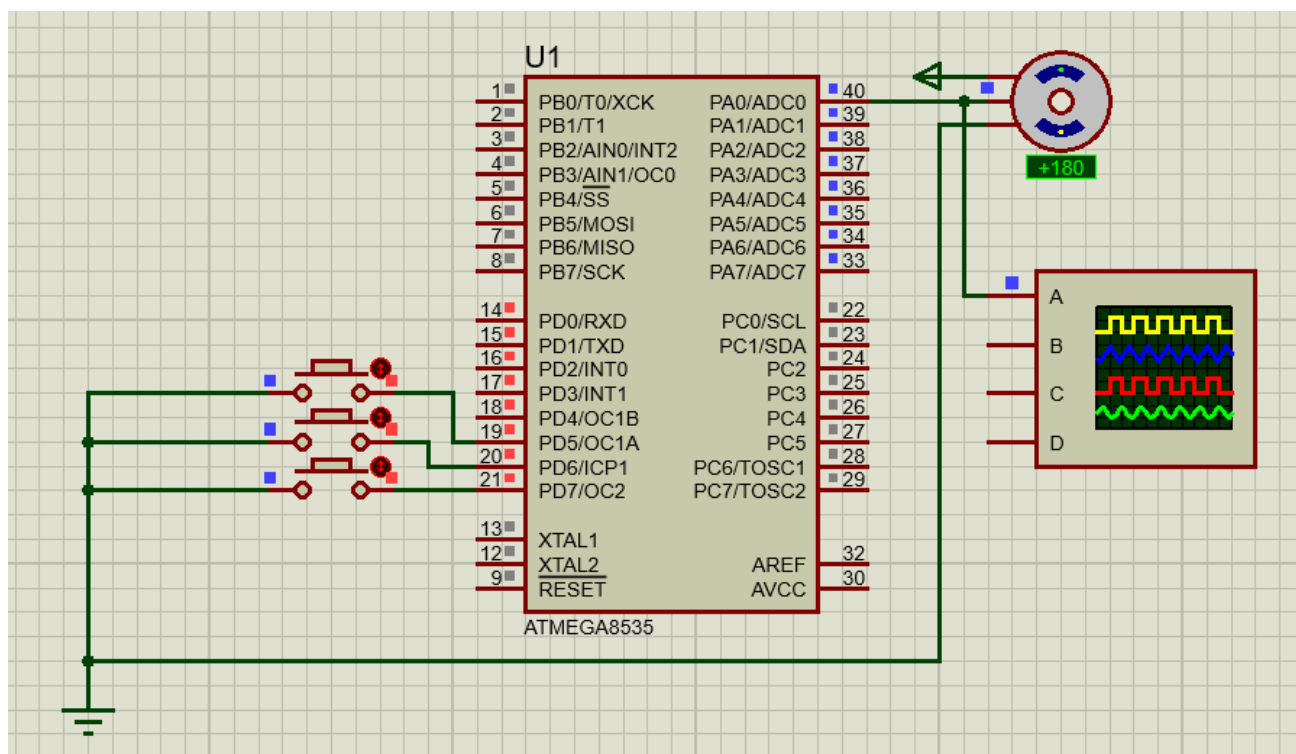
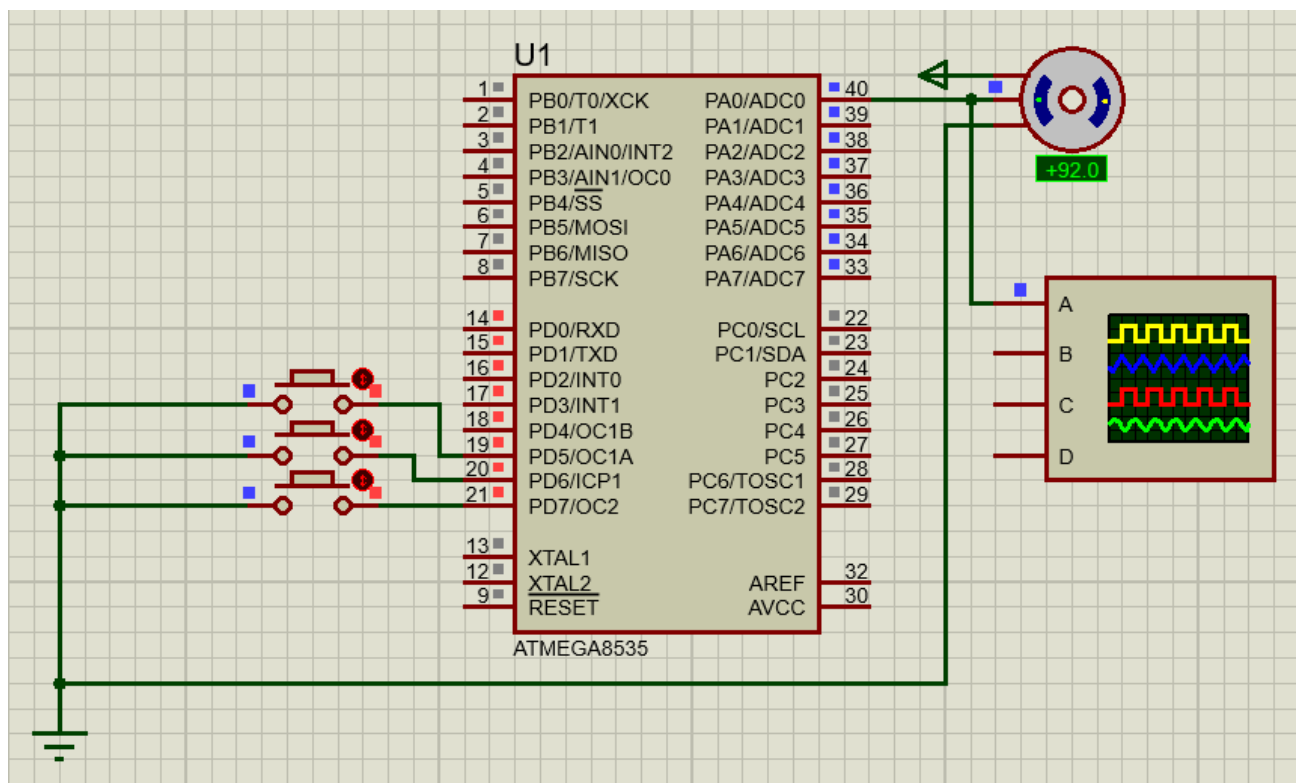


Ilustración 3. Simulación del funcionamiento en proteus.

Circuito Armado

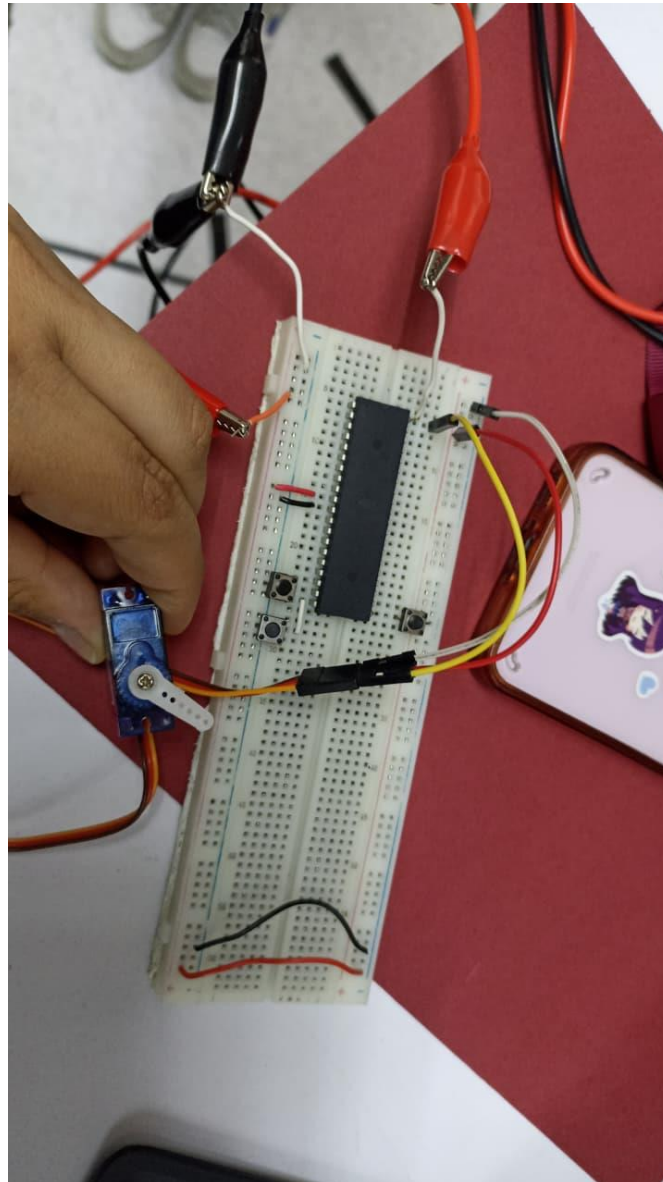


Ilustración 4. Armado físico del circuito funcionando.

Conclusiones

Ayala Fuentes Sunem Gizeht

A través de esta práctica, se logró comprender la importancia de la señal PWM para el control preciso de periféricos como el servomotor. Fue fundamental entender que, a diferencia de un motor de corriente continua convencional, el servomotor requiere una interpretación exacta de la duración del pulso eléctrico para definir su posición angular. La implementación exitosa del sistema permitió verificar que el diseño lógico cumple con el posicionamiento requerido en los ángulos de 0, 90 y 180 grados.

Guevara Badillo Areli Alejandra

El desarrollo de la práctica permitió analizar el funcionamiento de los retardos por software. La implementación de la rutina de retardo medms fue una pieza clave para controlar la longitud del tiempo en estado alto (*high*) y bajo (*low*) de la señal de control. Observamos que, al variar estos parámetros mediante la macro de pulso, es posible manipular de manera efectiva el ángulo de rotación del servomotor de acuerdo con las especificaciones del ejercicio.

León Pérez May Carolina

La integración del hardware y software permitió validar el control del servomotor mediante interfaces de entrada físicas como los botones pulsadores. Se configuró correctamente el Puerto A como salida para la señal de control y el Puerto D con resistencias *pull-up* para las entradas de los botones. Esta práctica demuestra la capacidad del ATmega8535 para gestionar eventos en tiempo real y realizar tareas de control de movimiento de forma estable y repetible.

Referencias

AdvancedMotionControls. (30 de Mayo de 2025). *Advanced Motion Controls*. Obtenido de Que es un servomotor: <https://www.a-m-c.com/es/servomotor/>

Ingenieria Mecafenix. (8 de Noviembre de 2022). *Ingenieria Mecafenix*. Obtenido de Que es un servomotor y como funciona: <https://www.ingmecafenix.com/electronica/motores-electronicos/servomotor/>

Naylamp Mechatronics. (7 de Enero de 2026). *Naylamp Mechatronics*. Obtenido de Tutorial uso de servomotor con Arduino: https://naylampmechatronics.com/blog/33_tutorial-uso-de-servomotores-con-arduino.html